

Buffer Overflows & Memory Corruption

The classic stack overflow

A function reads input into a fixed-size buffer and overruns the boundary, overwriting the saved return address.

```
char buf[64];
gets(buf);          // no bounds check!
// attacker sends 64 bytes + 4 of shoe + EIP = shellcode address
return;             // jumps to attacker's code
```

Anatomy of the exploit

```
[ shellcode ][ padding ][ saved EBP ][ saved EIP -> shellcode ]
```

NOP sled (\x90...) to tolerate address jitter

EIP overwritten with address of the sled

Shellcode = machine instructions (e.g., `execve /bin/sh`)

Modern protections

Defense	What it does	Typical bypass
NX/DEP	Data pages aren't executable	ROP: reuse existing instructions (gadgets)
ASLR	Randomize addresses	Info leak first; or brute small entropy; or non-PIE
Stack canaries	Canary before return addr; detect smash	Leak the canary; or overwrite before it
PIE	Relocate binary itself	Info leak of a module base

Defense	What it does	Typical bypass
RELRO	Harden GOT	Partial RELRO → GOT overwrite still possible
Fortify / stack protector	Compiler checks	Rarely bypassed without a leak

ROP in one line

Return-Oriented Programming: find **gadgets** (existing code sequences ending in `ret`), chain them so EIP walks through them – defeats NX by reusing the program's own code.

Why care (non-exploit roles too)

Blue team: know what "exploitable" means to size severity; want ASLR+NX+canary+PIE+RELRO defaults

Vuln researchers: use Fuzzing + dynamic binary analysis to find them

Detection: EDR flags unusual stack patterns, DEP-bypass behaviors

How to learn safely

Pwn/CTF academies (pwn.college, ROP Emporium) run on modern-hardened practice binaries. NEVER practice memory corruption on anything you don't own.

Memory corruption is the deepest well in offensive security. Start with pwn.college's beginner course; everything after is refinement.

← [Linux Privilege Escalation](#)

[Persistence & Lateral Movement...](#)